**Individual Reflection**

**CSC 481/581 - Game Engine Foundations - Fall 2026**

**Group 4: Sai Sumedh Kaveti**

**Milestone 4 - Networked Events**
**Project 4 - Individual Reflection**

**Individual Game Name: "Cave Ninja" GitHub Game Repository:**
https://github.com/CodeRockerr/CSC481-581-Group4/tree/main/Games/skaveti

## (A) My Role

I owned the Serialization, Server Routing, Queuing, De-duplication, Testing and Network Design Document. Adit implemented the local event infrastructure (queuing, priorities, replay). Yeva implemented gameplay events and input. I built on Adit's events and did not change his event types. I rebased on both of their merges. I converted Cave Ninja to use events. There were 4 gameplay events, one shared Death event and an offline replay (see Figures 1 – 6).

## (B) Engine Design: How Events Cross the Network

**Event Management (Team Design):** Events are made up of a value object containing a type id, event id (unique), time stamp from our timeline, priority, source, target and payload. With this system events can be placed into a queue, stored, serialised and interpreted on any machine. Listeners register for a particular event type and receive a token to deregister themselves. This means that producers don't have to know who their listeners are. The raise function only adds to the queue, which means there is no recursion. The main thread handles events at "safe" points in the frame, these are handled in order of their time stamp, then priority then insertion order. This Event Manager was designed by Adit. I designed the network layer based on this, and did not modify the event types he had.

**Why server-based:** Events can be shared via the server ensuring a single source of truth for ordering. Bad events can also be rejected at the server. We already had a server. Our player movements are simulated on the client so it made sense to use the existing request/reply pattern. This ensured fast responses to user input and no extra sockets/threads were required. If we had used a distributed approach we would have had to ensure ordering between clients without a referee. Events are simply queued at the network layer and handlers are always executed on the main thread.

**Design philosophy:** Events describe occurrences, and snapshots keep synchronizing positions. Each rule (death counting, respawn, invulnerability) has one owner. Producers do not know their listeners. These choices make it possible to add shared events, replay and server-side rules later without changing the protocol.

**Serialization:** Created EventCodec which can encode and decode the Event struct into both binary and text formats. This was done explicitly rather than using memory copy techniques. As the Event struct contains strings, a variant payload and a list of fields which have maximum sizes (strings can be up to 4096 bytes, fields can be up to 256), the decoder will reject corrupted data (too short), too long, type 0 and unknown tags, because the server is decoding data from untrusted clients. The origin and shareable fields are not transmitted as they represent where the event originated from on a single machine. If there are zero events within a packet the packet is identical in bytes to the Milestone 3 packet.

**Routing:** All events are sent by the client as part of the regular State packet (every ~16 ms). The server replies with these events in its Snapshot. This was simply re-used and no new sockets or threads were needed. For each accepted event the server duplicates it into a pending queue for every other client using a mutex to ensure events are seen in the same order. The owner id is then changed to that of the sender so clients cannot impersonate each other. Replay control events and those of type 0 are refused. Rules can be added to a game using setEventFilter. Server generated events can be broadcast using broadcastEvent.

**Identity and clocks:** A shared event names its object by (owner id, network id), which every client can resolve. After Adit's identity fix, NetEventBridge passes these ids through unchanged. Events that point at an offline object (owner id 0xFFFFFFFF) are never sent: the bridge keeps them local and the server rejects them. A received event is marked Origin::Network, is not shareable and gets timestamp -1, so the receiver stamps it with its own clock. This stops clock differences from changing event order and stops a received event from being sent again. Snapshots stay the only thing that creates remote objects, so a networked Spawn event must not create one. While a replay plays, the bridge discards network events and the game skips snapshot sync and publish.

## (C) Delivery Queues and Duplicate Prevention

Transport (ZeroMQ request reply) itself is only reliable while the connection is up. If a request times out then the client will retry which could result in duplicate events. A light reliability layer was added to both client links. This gives sequential numbering to all events, and both sides send cumulative acknowledgements of the highest in-order event they have seen. Events are held until acknowledged and all unacknowledged events (up to 64 per packet) are re-transmitted each time. The receiving side will only accept events with the exact expected number. Any lower number is a duplicate and will be ignored. If there is a gap then processing will be halted. This guarantees events are only ever delivered in order. The server also holds queues of pending events for each recipient and will only deliver an event once it has been acknowledged.

This guarantees events are delivered once and in order with no new protocol. However, this does cause bandwidth issues due to everything being re-transmitted. Design notes can be found in Docs/Design/Design_NetEvents.md

## (D) Testing and Results

net_events_test passes on my Mac build (1,538 checks), and all 17 ctest tests pass in a Release build after the rebase onto main (Figure 7).

| Area | What the test checks |
|---|---|
| Codec | Round trips in both formats; truncated data, trailing bytes and type 0 are rejected |
| Delivery | Loopback delivery in both formats, including a 150-event burst delivered in order and exactly once |
| Local-only rules | Non-shareable and replay events stay local; the server filter rejects events; events flow both ways; server broadcast works |
| Raw-socket attacks | Duplicates, gaps and a spoofed owner id sent straight at the server; redelivery until ack |
| Offline and replay | Offline references stay local; network events are dropped during playback |
| Disconnects | A client that leaves or times out does not affect the others |

**Mutation check:** I deliberately removed the duplication check resulting in the test failing. This demonstrates how the test prevents this from happening. I ran ThreadSanitizer on the stripped build and only got reports in libzmq and cout calls.

**Observations:** The number of deaths was equal on both clients. (6 deaths | 4 remote deaths and 4 deaths | 6 remote deaths) When using binary and text mode, when one client died the replicas on the other client turned red. There were two log entries for each death and the time between them was around 26 ms (12:36:50.325 and 12:36:50.351). The round trip time was 0.4 ms (Figures 1-3).

**Limits of this evidence:** Both the clients and the server were running on the same Mac and hence the 26 ms is likely to be mainly due to the polling interval (16 ms) and timing of the frames, rather than network delay. I didn't measure how much of this was due to the network. This does show however that there is enough speed in the pipeline for real game play. The tests don't account for packet loss or delay. The resend logic is tested by forcing duplicates and gaps, not by simulating a lossy network.

### (E) Cave Ninja: Using Events

Before, the game killed and respawned the ninja directly inside the update loop. Now each step is its own event with its own handler:

| Event | Raised by | Handled by | Shared |
|---|---|---|---|
| Input | Keys, each frame ("move", and "jump" on the first frame of the press) | Stores the move direction and a queued jump; applied on the next update (one frame of latency) | No |
| Collision | Overlap check with death zones, the shadow wisp and checkpoints, only when an overlap starts | Hazard or wisp raises Death; a checkpoint updates the checkpoint | No |
| Death | Collision handler, or falling below the world | Local: counts a death, starts 150 ms of invulnerability, raises Spawn. Network: flashes the replica red and counts a remote death | Yes |
| Spawn | Death handler | Moves the ninja to the last checkpoint | No |

The invulnerability window uses the engine clock (events.now()), not wall time, so one hazard cannot cause two deaths and the behavior is the same in replay. Moving the death counter into the Death handler fixed a real problem: the count used to rise in the respawn code, so any other path to respawning would have counted wrongly.

A shared Death names the ninja by (client id, network id). The ninja is created with a non-zero network id (its object id), because the engine treats a reference with network id 0 as offline and keeps that event local. I leave the event's source empty, because the cause is a local hazard or the wisp, which other machines cannot resolve. On the other client, the handler sees Origin::Network, flashes that player's replica red for 1.2 seconds and adds to a “remote deaths” counter in the title bar. I made no other event networked: position already arrives in snapshots, and sending Spawn would create duplicate objects.

**Replay.** I use Adit's ReplayController with two callbacks, one that captures the game state (ninja, moving platform paths, camera, checkpoint) and one that restores it. The state is stored in a std::any, so the engine does not need to know the game's types. R starts recording, T stops and Y plays back, and the title bar shows REC, REPLAY or live. On playback the ninja returns to the exact start position and the recorded input drives it (Figures 4-6). Replay is offline only, because the state callbacks are installed only when the game is not networked.

## (F) What Did Not Work at First

**A stacked PR went to the wrong branch:** PR #26 was stacked on #25. After #25 merged, #26 was merged into feat/net-events instead of main, so my game was not in main. I rebased on main after Adit's game (#27) and opened #29.

**Adit's identity change broke my Death event:** My Death used the target {0, playerId}, which no longer matched replicas once ids stopped being remapped. I fixed it by giving the ninja a non-zero network id, naming the target with its real identity and leaving the source empty.

**A rebase conflict:** The root CMakeLists.txt conflicted with Yeva's merge. I kept both test blocks.

**A wrong test run:** I once ran an old build because the patch was not in the repo folder. I re-applied it and rebuilt.

**A bug in my own disconnect test:** Client B stayed silent during a 5 second wait and timed out (the server timeout is 600 ms). I fixed it by polling B inside the wait.

**A mutation test that proved nothing:** Removing the bridge check was hidden by the server's own rejection of the same events. I redid it by removing the whole bridge check, and the test then failed as it should.

**Unexplained log lines:** Some Death events with the reason fell appeared in the logs when I did not expect them. I did not investigate this, so I cannot rule out an edge case in the below-the-world check.

## (G) Limitations and What It Enables

**Limitations:**

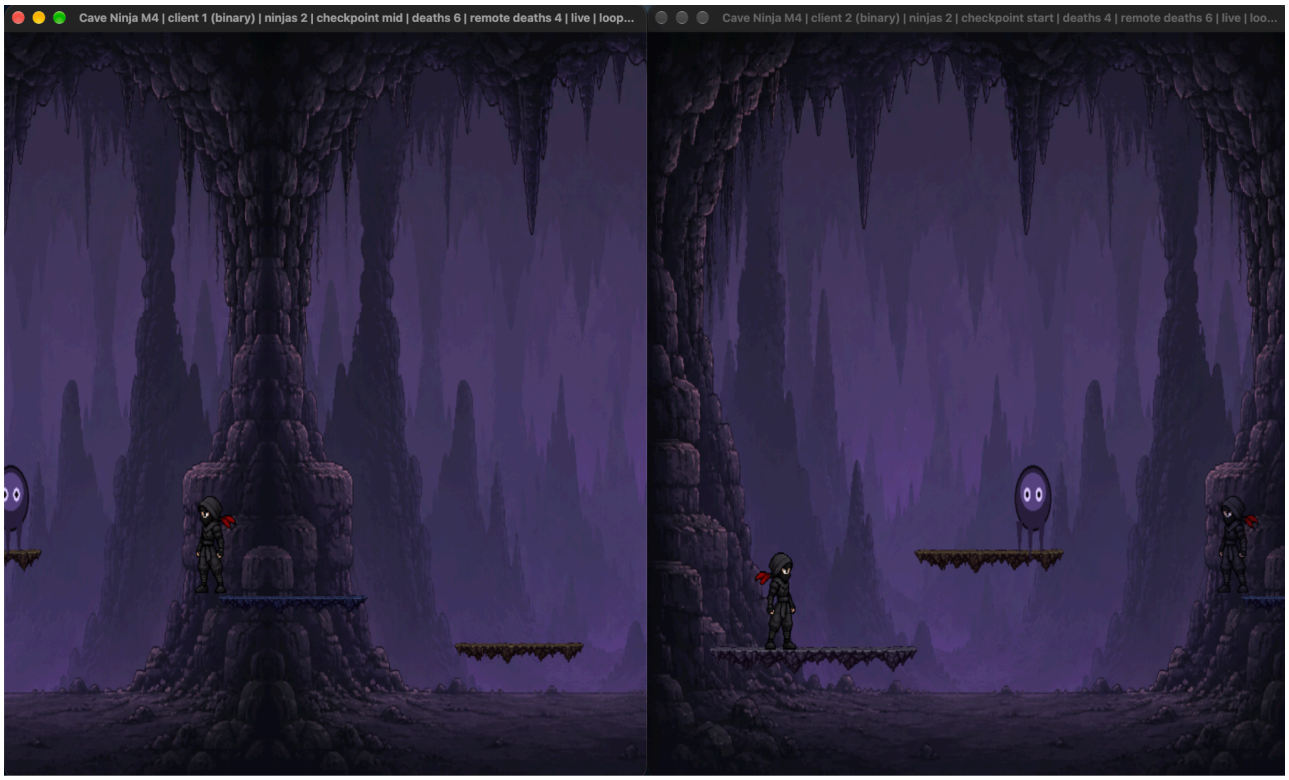
- Only one common event: death. Jump, collect and checkpoint events are local events.
- No online replay capability. Events can't be played back when using the event system over a network. This means I don't have a replay option available when playing with two players.
- There's a 1-frame input lag. Events are queued and applied on the next update.
- Landing and ground detection are coded directly.
- The sender will send all unacknowledged events. If there is high packet loss this is inefficient. It would be better if it could only send certain events.
- No interest filtering. Only 64 events allowed per packet. All clients receive all shared events.
- If a client times out then they lose the events in their pending queue and won't receive events which were sent whilst they were offline.
- Server authority is simply a hook. The owner id will be set by the server and any event which is trying to manipulate replay will be denied. Game logic will only be performed if the game has set an event filter.
- No events in the P2P mode.
- My game is merged into main as PR #29. It adds its own events for the gameplay (hazard, shadow wisp, fall, spawn).

**What it Enables:** new event types with just a type, payload and handler without any changes to the protocol. Game logic can be added to the server by way of the event filter. Selective resend and interest filtering can be tried on top of the existing sequence numbers and acks.

### (H) Attribution

Adit built the local event system (Event, EventManager, ReplayController), the identity fix merged in main, and his own game. Yeva built the gameplay events and input system that the shared code builds on. I wrote EventCodec, NetEventBridge, the server routing, sequence numbers and acks, the offline-reference and replay rules, net_events_test, Docs/Design/Design_NetEvents.md and Cave Ninja (Games/skaveti). The networking uses libzmq 4.3.5 and rendering uses SDL3.

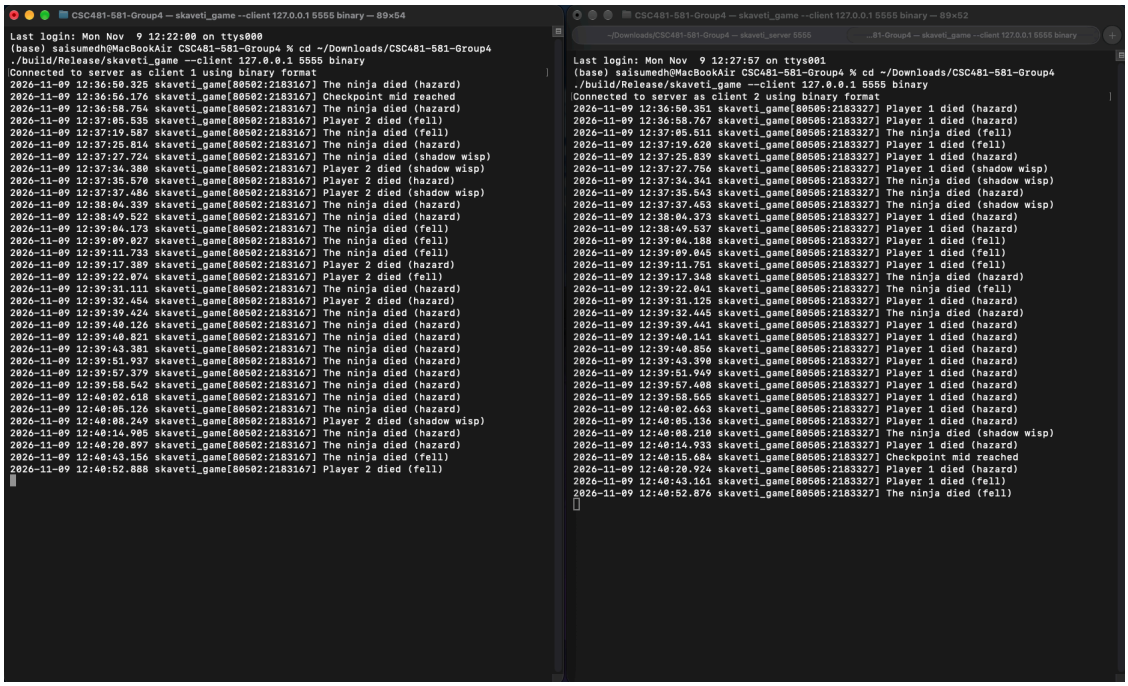
## Appendix - Screenshots



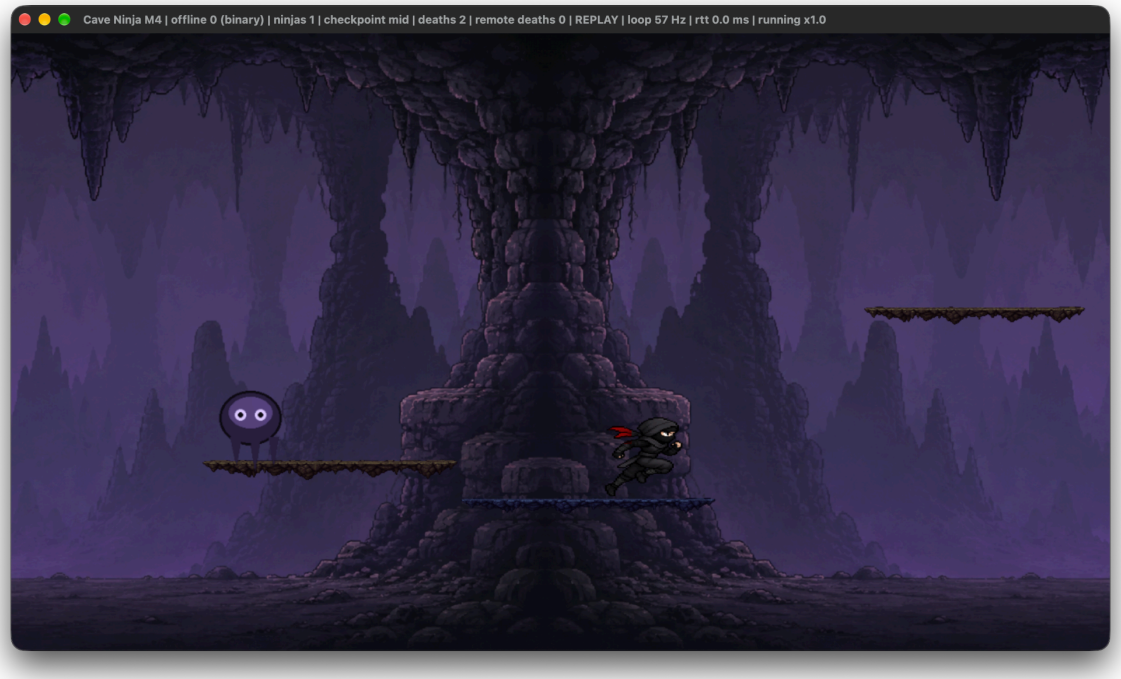

*(fig. 1 - Two clients side by side: client 1 shows deaths 6 | remote deaths 4 and client 2 shows deaths 4 | remote deaths 6, so both sides agree.)*



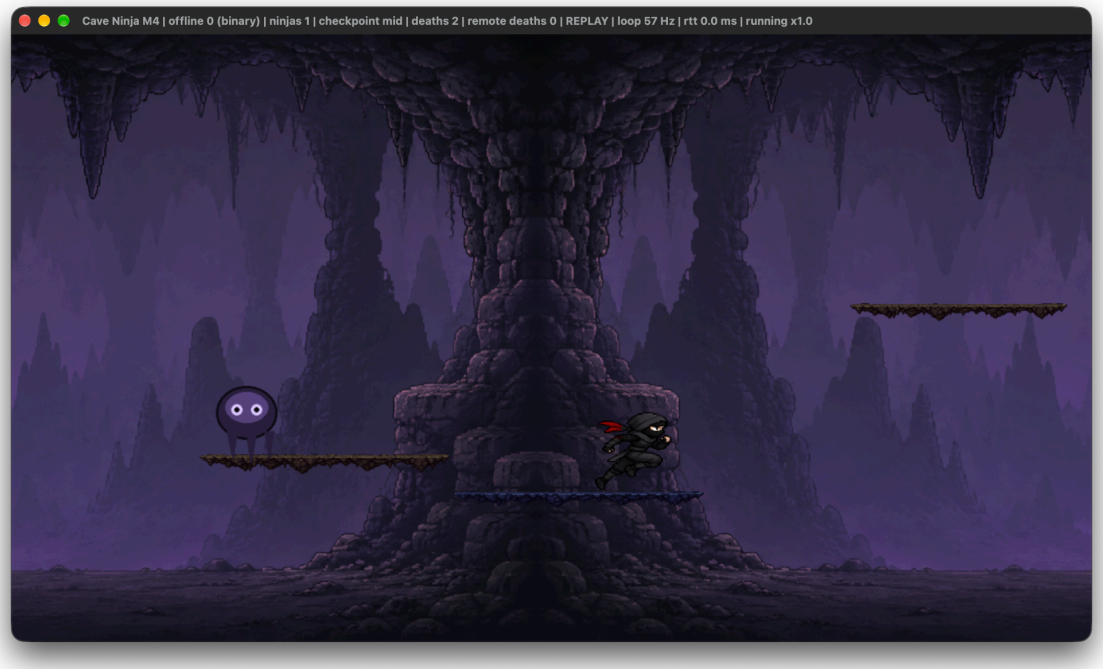

*(fig. 2 - Client 2's window right after a death in client 1: the replica ninja flashes red (remote deaths 22 from a longer run than fig. 1))*



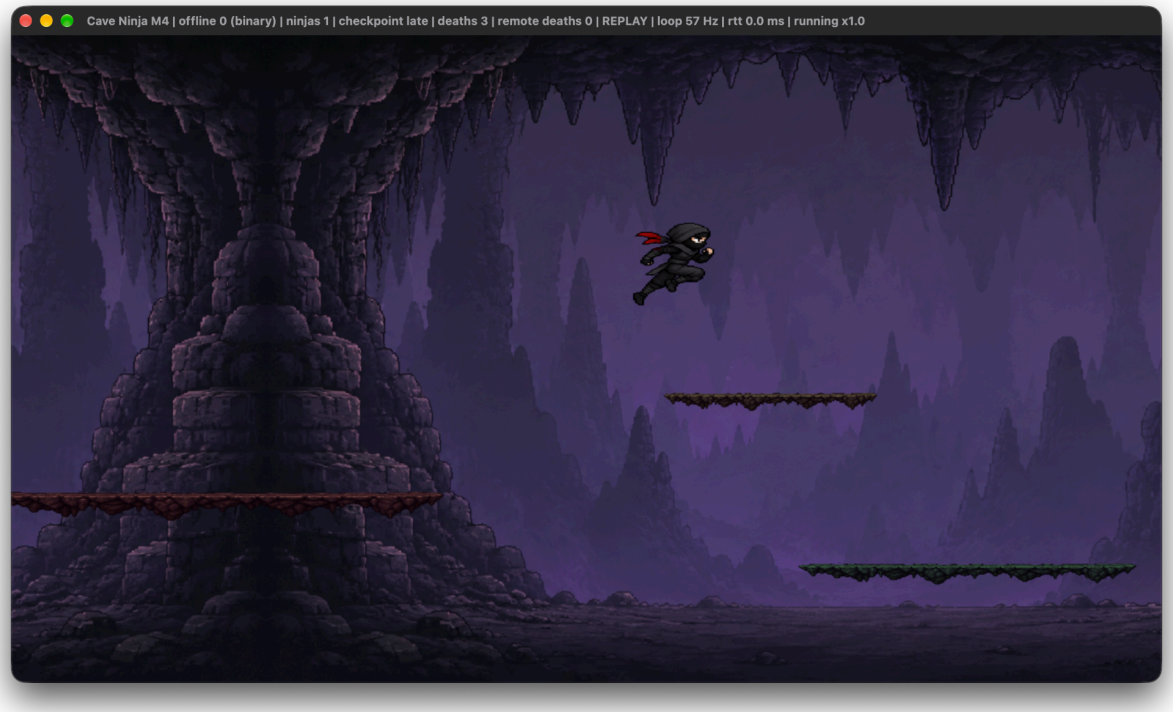

*(fig. 3 - Both terminals' event logs: The ninja died (...) on one side and Player N died (...) on the other, for the same deaths.)*




*(fig. 4 - Offline game with the title bar showing REC while recording.)*

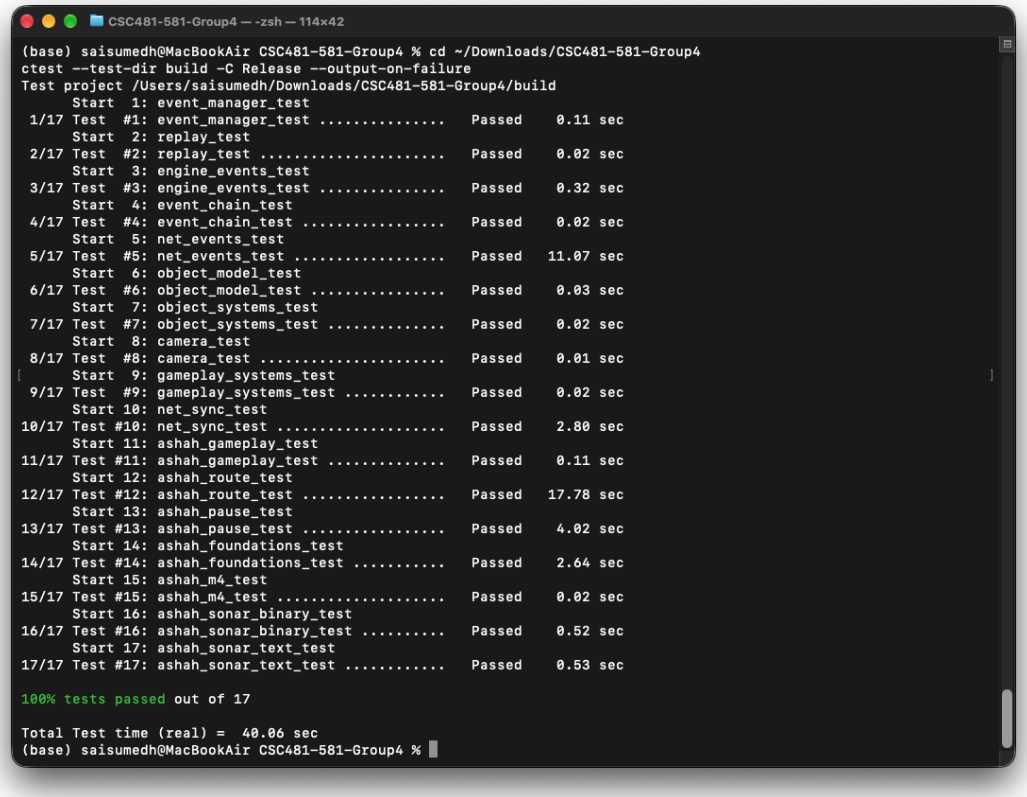



*(fig. 5 - Title bar showing REPLAY, with the recorded run playing back.)*

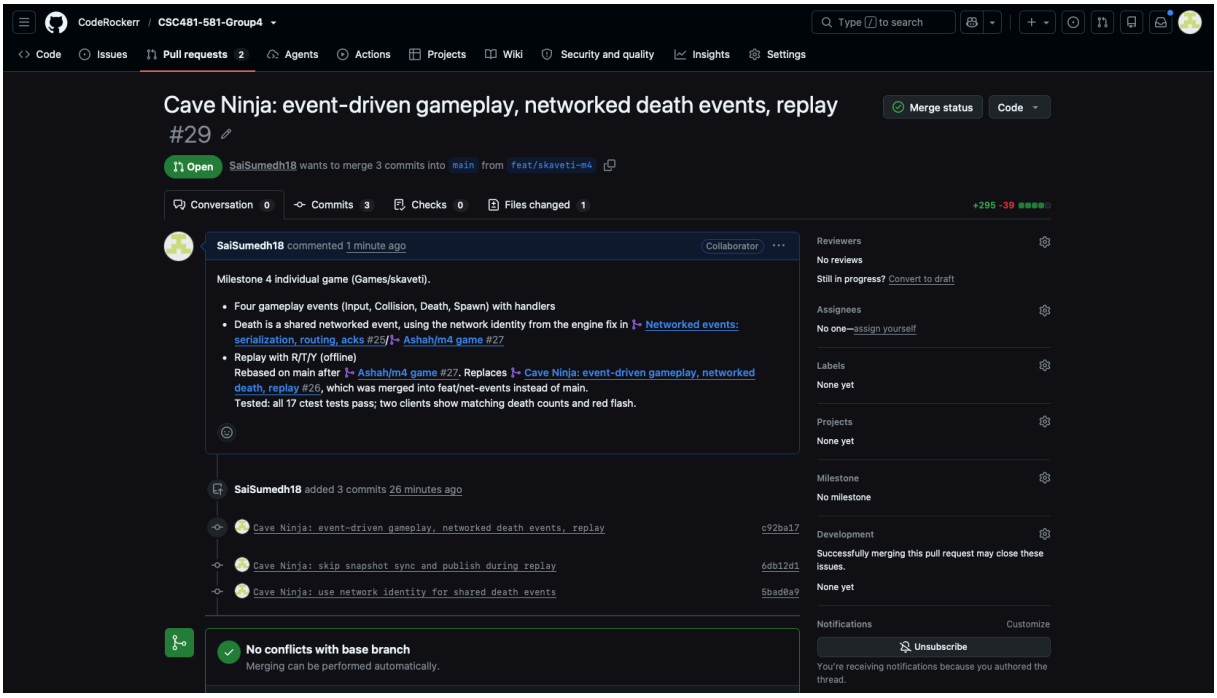



*(fig. 6 - REPLAY mid-jump, showing the recorded input driving the ninja.)*

```
CSC481-581-Group4 — -zsh — 85×27
Client 4 connected (2 connected)
Client 3 left (1 connected)
Client 4 left (0 connected)
Server listening on port 5615
Client 1 connected (1 connected)
Client 2 connected (2 connected)
Client 3 connected (3 connected)
Client 1 left (2 connected)
Client 3 timed out (1 connected)
Client 4 connected (2 connected)
Client 2 left (1 connected)
Client 4 left (0 connected)
Server listening on port 5616
Client 1 connected (1 connected)
Connected to server as client 1 using binary format
Client 2 connected (2 connected)
Connected to server as client 2 using binary format
Client 2 left (1 connected)
Client 1 left (0 connected)
Server listening on port 5617
Client 1 connected (1 connected)
Connected to server as client 1 using text format
Client 2 connected (2 connected)
Connected to server as client 2 using text format
Client 2 left (1 connected)
Client 1 left (0 connected)
net_events_test passed (1538 checks)
```




*(fig. 7 - ctest in a Release build: 100% tests passed out of 17, after the rebase onto main.)*




*(fig. 8 - PR #29 on GitHub: 3 commits into main, 1 file changed, no conflicts.)*